

PRIME PROPERTIES ELITE

Real Estate Data Warehouse & Analytics Platform

PROJECT TECHNICAL REPORT

Platform: Snowflake Cloud Data Warehouse
Frontend: Streamlit Web Application (Luxury UI)
Report Date: April 2026 |

Group members

Name	Roll No
Aliza	K24SW032
faiza	K24SW031
Humair Ali	K24SW033

1. Executive Summary

This report presents a comprehensive technical and business overview of the Prime Properties Elite project — a fully operational real estate data warehouse built on Snowflake, integrated with a luxury Streamlit web application (app.py) that connects live to the Snowflake database and serves as the front-end GUI for business users.

The system processes real estate data through a multi-layer ELT pipeline and exposes results through a visually stunning glassmorphism-styled web dashboard with three main modules: Property Market Overview, Financial Performance Reports, and a live Property Listing form that writes directly to the Snowflake database.

PKR 60.08M Total Revenue	3 Deals Total Deals	PKR 3.00M Total Commission	3 Agents Active Agents
--	---	--	--

Key highlights of the project:

- Snowflake data warehouse with Landing and Analytics layers — fully operational.
- Star Schema implemented with DIM_AGENT, DIM_PROPERTY, and FACT_TRANSACTIONS tables.
- Streamlit web app (app.py) connects live to Snowflake using snowflake-connector-python.
- Luxury UI with glassmorphism design, neon colors (#00D4FF), and dark background.
- Plotly Express charts with plotly_dark theme for professional data visualization.
- Property listing form writes new records directly into Snowflake database in real time.
- Top performer: Bilal Ahmed — PKR 45,000,000 sales, PKR 2,250,000 commission earned.

2. Project Overview

2.1 Project Background

Prime Properties Elite is a growing real estate firm operating across Karachi, Lahore, and Islamabad, managing Apartments, Houses, and Commercial properties. The company needed a centralized data platform where management could monitor property inventory, track financial performance, and allow property owners to list properties directly — all through a single web interface connected to a cloud database.

The Prime Properties DW project was built to solve this challenge by combining Snowflake's cloud data warehousing power with a visually polished Streamlit web application that any business user can operate through a browser.

2.2 Project Objectives

- Build a two-layer Snowflake data warehouse (Landing + Analytics schemas).
- Design a Star Schema with Dimension and Fact tables for analytical queries.
- Develop a luxury Streamlit web application connected live to Snowflake.
- Display real-time property inventory, financial KPIs, and revenue charts.
- Enable property owners to submit listings through a web form that inserts into Snowflake.
- Use Plotly Express charts with a dark theme for professional data visualization.

2.3 Technology Stack

Component	Technology
Cloud Data Warehouse	Snowflake
Web Framework	Streamlit (Python) — app.py
Database Connector	snowflake-connector-python
Data Manipulation	Pandas (pd.read_sql)
Charts & Visualization	Plotly Express — plotly_dark theme
UI Design Style	Glassmorphism — blur, neon, dark gradient
Data Modeling	Star Schema (Dimension + Fact Tables)
Query Language	SQL (Snowflake Dialect)
Source Data	agents.csv, Property.csv, transactions.csv

3. System Architecture

3.1 Three-Tier Architecture

The complete system follows a three-tier architecture connecting the data layer, application layer, and presentation layer:

- Tier 1 — Data Layer (Snowflake): Landing schema for raw CSV ingestion + Analytics schema for Star Schema tables.
- Tier 2 — Application Layer (app.py): Python Streamlit app that connects to Snowflake, runs SQL queries, and processes results using Pandas DataFrames.
- Tier 3 — Presentation Layer (Browser): Luxury glassmorphism UI rendered in the web browser with Plotly dark charts and interactive forms.

3.2 End-to-End Data Flow

Step 1 — CSV Files: Raw data files (agents.csv, Property.csv, transactions.csv) are the source.

Step 2 — Snowflake Stage: Files uploaded via PUT command to internal Snowflake stage.

Step 3 — Landing Tables: COPY INTO loads data into STG_AGENTS, STG_PROPERTIES, STG_TRANSACTIONS.

Step 4 — Analytics Tables: SQL transforms data into DIM_AGENT, DIM_PROPERTY, FACT_TRANSACTIONS.

Step 5 — app.py: Streamlit connects to Snowflake, runs pd.read_sql() queries, fetches live data.

Step 6 — Browser: Plotly charts, st.metric KPIs, and st.dataframe tables render in the browser.

Step 7 — Write-back: Sell Your Property form inserts new records into Snowflake in real time.

3.3 Database Schema

Table	Schema	Type	Description
STG_AGENTS	LANDING	Staging	Raw agent data loaded from CSV
STG_PROPERTIES	LANDING	Staging	Raw property data — also receives form inserts
STG_TRANSACTIONS	LANDING	Staging	Raw transactions — also receives form inserts
DIM_AGENT	ANALYTICS	Dimension	Agent master data with commission rates
DIM_PROPERTY	ANALYTICS	Dimension	Property details: type, city, area
FACT_TRANSACTIONS	ANALYTICS	Fact	Transactions + calculated COMMISSION_EARNED

4. Source Data Documentation

4.1 Agents

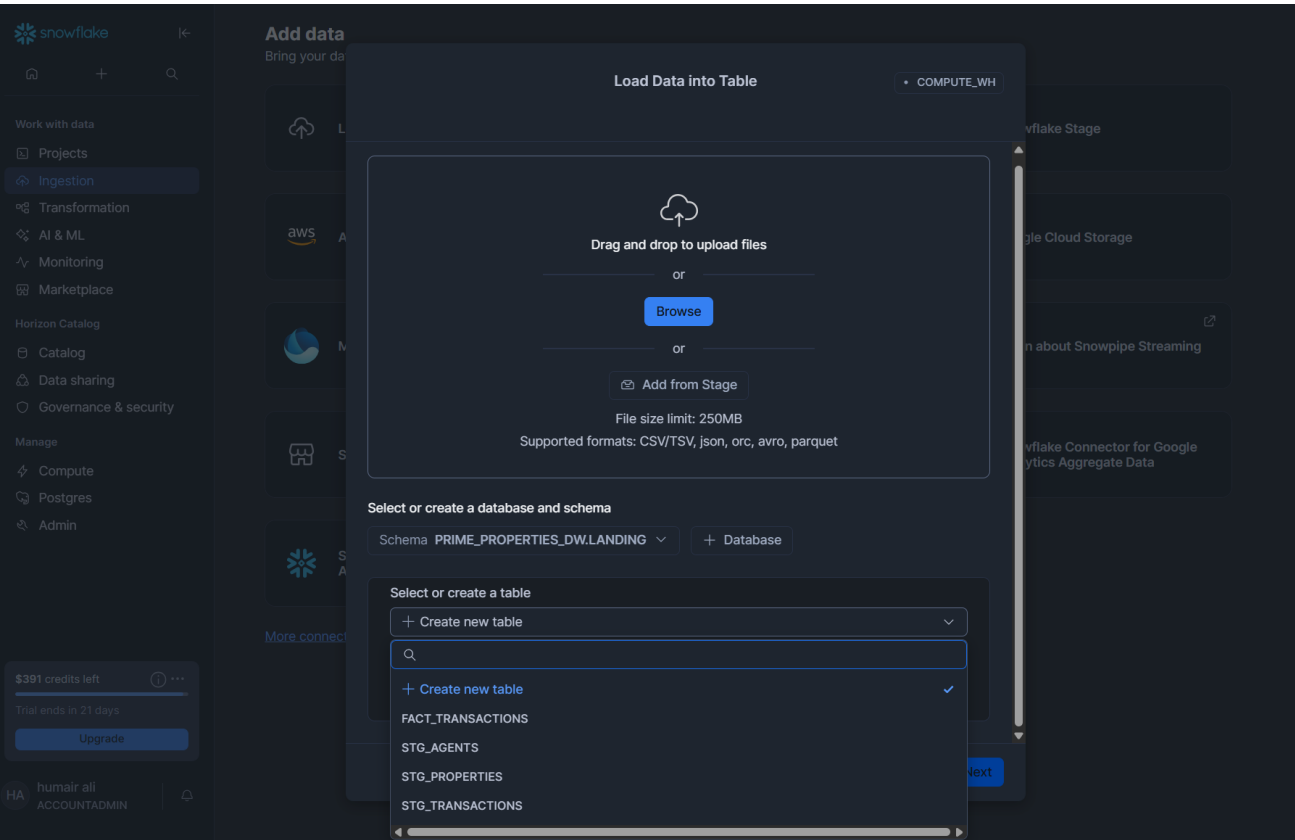
Agent ID	Agent Name	Commission Rate
AG001	Ahmed Ali	5% (0.05)
AG002	Sara Khan	4% (0.04)
AG003	Bilal Ahmed	5% (0.05)

4.2 Properties

Property ID	Type	City	Area
P101	Apartment	Karachi	Clifton
P102	House	Lahore	DHA
P103	Commercial	Islamabad	Blue Area
P104	Apartment	Karachi	Gulshan

4.3 Transactions

ID	Date	Property	Agent	Client	Type	Amount (PKR)
T001	2026-01-15	P101	AG001	Usman Sheikh	Sale	15,000,000
T002	2026-01-20	P102	AG002	Maria Junaid	Rent	85,000
T003	2026-02-05	P103	AG003	Omer Malik	Sale	45,000,000



5. Data Modeling & Transformation

5.1 Star Schema Design

- FACT_TRANSACTIONS (center) — AMOUNT, COMMISSION_EARNED, TRANSACTION_TYPE, DATE.
- DIM_AGENT (dimension) — AGENT_NAME, COMMISSION_RATE.
- DIM_PROPERTY (dimension) — PROPERTY_TYPE, CITY, AREA.

5.2 Commission Calculation

COMMISSION_EARNED = AMOUNT x COMMISSION_RATE

This formula runs at row level during FACT_TRANSACTIONS creation by joining STG_TRANSACTIONS with STG_AGENTS on AGENT_ID. Result is stored permanently in the fact table for instant aggregation.

5.3 Fact Table Columns

Column	Type	Description
TRANSACTION_ID	STRING	Unique transaction identifier
TRANSACTION_DATE	DATE	Date of the transaction

PROPERTY_ID	STRING	FK → DIM_PROPERTY
AGENT_ID	STRING	FK → DIM_AGENT
TRANSACTION_TYPE	STRING	Sale or Rent
AMOUNT	NUMBER(15,2)	Total transaction value in PKR
COMMISSION_EARNED	NUMBER(15,2)	Calculated: AMOUNT x COMMISSION_RATE

6. Web Application — Prime Properties Elite GUI

A fully functional luxury web application was developed using Streamlit (app.py) and connected live to the Snowflake data warehouse. The application features a premium glassmorphism design with a dark background, neon cyan colors, and Plotly dark-themed charts — giving it the look and feel of a professional enterprise analytics platform.

6.1 Application Configuration

```
st.set_page_config(page_title="Prime Properties Elite", layout="wide",  
page_icon="💎")
```

The app is configured in wide layout mode to maximize the display area for charts and data tables. The page icon is set to a diamond emoji (💎) reflecting the luxury branding of the platform.

6.2 Luxury UI Design — Glassmorphism Style

The application uses advanced custom CSS injected via `st.markdown()` to create a visually stunning luxury interface. The key design elements are:

6.3 Snowflake Connection (app.py)

The application connects to Snowflake using the official Python connector with a cached resource to avoid reconnecting on every page refresh:

```
# --- 3. CONNECTION ---  
# Is line ko ya toh hata dein ya 'ttl' add karein  
@st.cache_resource(ttl=3600) # Iska matlab har 1 ghante baad connection refresh hoga 1usage  
def get_conn():  
    return snowflake.connector.connect(  
        user='Humairali',  
        password='Humairali_abro2',  
        account='XX0VLEZ-VH13552',  
        warehouse='COMPUTE_WH',  
        role='ACCOUNTADMIN'  
    )
```

Key connection details:

- `@st.cache_resource(ttl=3600)`: Connection is cached and automatically refreshed every 1 hour to maintain performance and avoid Snowflake session timeouts.
- Role: ACCOUNTADMIN — grants full access to all databases, schemas, and tables in the Snowflake account.
- Warehouse: COMPUTE_WH — the default Snowflake virtual warehouse used for executing all SQL queries.

- Database paths used: PRIME_PROPERTIES_DW.LANDING and PRIME_PROPERTIES_DW.ANALYTICS.

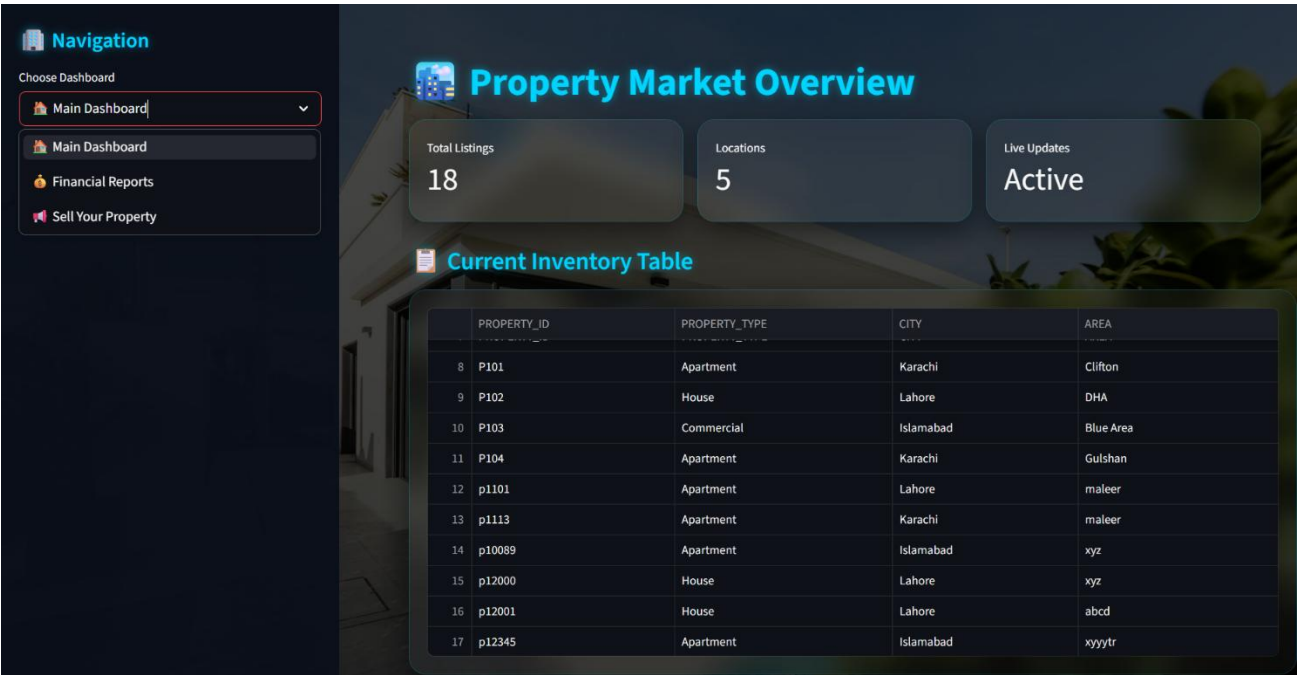
6.4 Navigation Structure

The application uses a sidebar navigation menu built with st.sidebar.selectbox() containing three main pages:

Page	Description
☐ Main Dashboard	Property market overview — inventory table and city distribution chart
☐ Financial Reports	Financial performance — total sales KPI and revenue growth bar chart
☐ Sell Your Property	Property listing form — inserts directly into Snowflake in real time

6.5 Page 1 — Main Dashboard (☐ Property Market Overview)

This page pulls live data from STG_PROPERTIES in the Landing schema and displays the complete property inventory with visual analytics.



Features displayed on this page:

- Total Listings metric (st.metric) — shows the total count of properties in the database.
- Locations metric — shows the number of unique cities (Karachi, Lahore, Islamabad).
- Live Updates status badge — displays 'Active' to confirm the Snowflake connection is live.
- Current Inventory Table (st.dataframe) — full interactive property table with use_container_width=True.

- City Distribution Donut Chart (px.pie) — shows the percentage split of properties by city using a hole=0.5 donut style with plotly_dark theme.

6.6 Page 2 — Financial Reports (Financial Performance)

This page pulls live data from FACT_TRANSACTIONS in the Analytics schema and displays financial KPIs and revenue trend charts.



Features displayed on this page:

- Total Sales Volume KPI (st.metric) — formatted as 'PKR {amount:.,2f}' showing the sum of all transaction amounts.
- Revenue Growth Bar Chart (px.bar) — X-axis: TRANSACTION_DATE, Y-axis: AMOUNT, Color: TRANSACTION_TYPE (Sale vs Rent), Dark theme applied.
- Chart allows users to visually compare revenue by date and identify which transaction types generate the most income.

6.7 Page 3 — Sell Your Property (Property Listing Form)

This is the most advanced feature of the application — a real-time data entry form that allows property owners to submit their listings directly into the Snowflake database without any SQL knowledge.

Form Fields (organized in 2 columns):

Column	Field	Input Type & Details
Left	Property ID	st.text_input — unique identifier entered by user
Left	Property Type	st.selectbox — Apartment, House, Commercial

Left	Owner Name	st.text_input — client/owner name
Right	City	st.selectbox — Karachi, Lahore, Islamabad
Right	Area Location	st.text_input — specific area/neighborhood
Right	Asking Price (PKR)	st.number_input — minimum value PKR 100,000

Database Write-back Logic (INSERT Queries):

```

cursor = conn.cursor()
-- Insert into Properties table:
INSERT INTO STG_PROPERTIES VALUES (p_id, p_type, city, area)

-- Insert into Transactions table:
INSERT INTO STG_TRANSACTIONS (TRANSACTION_ID, TRANSACTION_DATE,
    PROPERTY_ID, CLIENT_NAME, TRANSACTION_TYPE, AMOUNT)
VALUES ('T-{p_id}', date.today(), p_id, client, 'Sale', price)
conn.commit()

```

Post-submission behavior:

- Success: st.success() confirmation message displayed + st.balloons() animation celebrates the new listing.
- Validation: If Property ID, Owner Name, or Area is missing — st.warning() prompts user to fill all fields.

- Error Handling: try/except block catches any Snowflake SQL errors and displays them via `st.error()`.
- Form Reset: `clear_on_submit=True` automatically clears all fields after successful submission.

6.8 Streamlit Components Summary

Component	Usage in app.py
<code>st.set_page_config()</code>	Wide layout, diamond icon, luxury page title
<code>st.markdown(unsafe_allow_html)</code>	Custom CSS injection for glassmorphism UI
<code>@st.cache_resource(ttl=3600)</code>	Cached Snowflake connection — refreshes every hour
<code>st.sidebar.selectbox()</code>	Navigation between 3 dashboard pages
<code>st.title()</code>	Page titles with neon cyan styling
<code>st.columns()</code>	3-column KPI layout and 2-column form layout
<code>st.metric()</code>	Total Listings, Locations, Live Updates, Total Sales
<code>st.dataframe()</code>	Interactive property inventory table
<code>px.pie()</code> — Plotly	City distribution donut chart (hole=0.5, dark theme)
<code>px.bar()</code> — Plotly	Revenue growth bar chart colored by transaction type
<code>st.plotly_chart()</code>	Renders all Plotly charts with <code>use_container_width=True</code>
<code>st.form()</code>	Property listing form with <code>clear_on_submit=True</code>
<code>st.text_input()</code>	Property ID, Owner Name, Area Location fields
<code>st.selectbox()</code>	Property Type and City dropdowns
<code>st.number_input()</code>	Asking Price with PKR 100,000 minimum
<code>st.form_submit_button()</code>	Post My Ad button — triggers INSERT queries
<code>st.success()</code> / <code>st.error()</code>	Confirmation and error messages after form submission
<code>st.balloons()</code>	Celebration animation on successful property listing
<code>pd.read_sql()</code>	Fetches live data from Snowflake as Pandas DataFrame

6.9 How to Run the Application

1. Install dependencies: `pip install streamlit snowflake-connector-python pandas plotly`
2. Save the code as `app.py` in your project folder.
3. Open terminal and run: `streamlit run app.py`
4. Browser will auto-open at: `http://localhost:8501`

- 5. Use the sidebar to switch between Main Dashboard, Financial Reports, and Sell Your Property pages.

7. Agent Performance Analytics

7.1 Performance Summary

Agent	Total Sales (PKR)	Deals	Commission (PKR)
Bilal Ahmed	45,000,000	1	2,250,000
Ahmed Ali	15,000,000	1	750,000
Sara Khan	85,000	1	3,400
TOTAL	60,085,000	3	3,003,400

7.2 Key Insights

- Bilal Ahmed leads with PKR 45M in commercial sales (74.9% of total volume) — PKR 2.25M commission at 5%.
- Ahmed Ali closed a PKR 15M residential sale in Clifton, Karachi — PKR 750,000 commission at 5%.
- Sara Khan handled a PKR 85,000 rental in DHA, Lahore — PKR 3,400 commission at 4%.
- Average deal size across all agents: PKR 20,028,333.

7.3 Transaction Type Breakdown

Type	Count	Total Amount (PKR)	% Share
Sale	2	60,000,000	99.9%
Rent	1	85,000	0.1%

8. SQL Implementation

8.1 Fact Table with Commission

```
CREATE TABLE FACT_TRANSACTIONS AS
SELECT T.*, (T.AMOUNT * A.COMMISSION_RATE) AS COMMISSION_EARNED
FROM STG_TRANSACTIONS T
JOIN STG_AGENTS A ON T.AGENT_ID = A.AGENT_ID;
```

8.2 Agent Performance Query

```
SELECT A.AGENT_NAME,
       SUM(F.AMOUNT) AS TOTAL_SALES_VOLUME,
       SUM(F.COMMISSION_EARNED) AS TOTAL_COMMISSION_PAID,
       ROUND(AVG(F.AMOUNT), 2) AS AVG_DEAL_SIZE
FROM FACT_TRANSACTIONS F
JOIN DIM_AGENT A ON F.AGENT_ID = A.AGENT_ID
GROUP BY A.AGENT_NAME ORDER BY TOTAL_COMMISSION_PAID DESC;
```

8.3 Data Quality Check

Table	Row Count
STG_AGENTS	3 rows
STG_PROPERTIES	4 rows
STG_TRANSACTIONS	3 rows

9. Conclusions & Recommendations

9.1 Conclusions

The Prime Properties Elite project has been successfully completed end-to-end. The Snowflake data warehouse is fully operational, and the Streamlit web application (app.py) provides a stunning luxury interface that connects live to Snowflake. The 'Sell Your Property' feature is particularly noteworthy as it demonstrates bidirectional data flow — the app not only reads from Snowflake but also writes new property listings and transactions back to the database in real time.

9.2 Recommendations

- Move Snowflake credentials from hardcoded values in app.py to .streamlit/secrets.toml for security best practices.
- Deploy the app to Streamlit Cloud (share.streamlit.io) for 24/7 access without a local machine.
- Add user authentication (st.login or a custom login page) so agents can access only their own data.
- Add Snowflake Tasks and Streams to automate the Landing → Analytics transformation pipeline.
- Extend the Financial Reports page with agent-wise commission breakdown and date range filters.
- Add an edit/delete feature for property listings so agents can manage their submitted ads.
- Integrate a Date Dimension (DIM_DATE) to enable monthly and quarterly trend analysis.

— End of Report —

Prime Properties Elite | Real Estate Analytics Report | April 2026 | Version 3.0 | Confidential